# Lab Overview & Objectives

**Objective: To deploy a Network Intrusion Detection System (NIDS) using Snort 2.x to detect common reconnaissance and scanning activities such as ICMP ping sweeps and Nmap scans.**

### Important Note on Software Versions

This practical is specifically designed for:
**Snort Version:** Snort 2.x (typically 2.9.x)
**Operating System:** Ubuntu 24.04 LTS or earlier (Newer Ubuntu versions like 25.10 do not include Snort in default repositories)

## 1. Lab Components

1. Attacker: Kali Linux (The threat actor initiating scans).
2. Target: Metasploitable 2 (The vulnerable victim server).
3. NIDS Sensor: Ubuntu (The Snort machine sniffing traffic).

### Example Network Configuration

All three machines should exist on the same virtual network segment.

| Component | IP (Example) | Role | Key Configuration |
|---|---|---|---|
| **Kali Linux** | 10.0.2.2 | **Attacker** | Initiates scans and attacks. |
| **Metasploitable** | 10.0.2.5 | **Target** | The vulnerable victim. |
| **Ubuntu Server** | 10.0.2.10 | **NIDS Sensor** | **Snort** is installed here; **Promiscuous Mode** enabled. |

## 2. Snort Rule Structure: Header and Options

Every Snort rule is a single line of text with two main parts: the **Rule Header** and the **Rule Options**.

#### 2.1. The Rule Header (What, Where, and When)

The rule header is the first part of the rule and defines the action to take, the protocol, the source/destination IPs, and the ports.

| Component | Description | Example |
|---|---|---|
| **Rule Action** | The action Snort performs when the rule is triggered. | alert |
| **Protocol** | The network protocol the rule applies to. | tcp, udp, icmp, ip |
| **Source IP** | The IP address of the sender. | $EXTERNAL_NET, 192.168.1.10, any |
| **Source Port** | The port number of the sender. | any, 1024: (ports $> 1024$) |
| **Direction Operator** | Defines the flow of traffic. | -> (Unidirectional) or <> (Bidirectional) |
| **Destination IP** | The IP address of the receiver. | $HOME_NET, 10.0.2.5, any |
| **Destination Port** | The port number of the receiver. | 80, 22, any |

**Rule Action Details:**

| Action | Description |
|---|---|
| **alert** | Generates an alert and logs the packet. (Most common for NIDS). |
| **log** | Logs the packet without generating an alert. |
| **pass** | Ignores the packet, stopping inspection on it. (Used in IPS mode to allow known traffic). |
| **drop** | Blocks (drops) the packet and logs it. (Used in IPS mode). |

**Direction Operator Details:**

| Operator | Meaning |
|---|---|
| **->** | Traffic flows from the Source to the Destination (Unidirectional). |

| | |
|---|---|
| <> | Traffic flows in either direction (Bidirectional). |

### 2.2. The Rule Options (Why and How)

The rule options are enclosed in parentheses () and provide details about *why* the rule should fire and how the alert should be handled. Options are separated by semicolons ;.

| Option Class | Option Keyword | Description | Example |
|---|---|---|---|
| **General** | **msg** (Mandatory) | The text that appears in the alert log. | msg:"FTP Login Attempt"; |
| | **sid** (Mandatory) | **Snort ID**; a unique number for the rule. Custom IDs should start at **1,000,000** or higher. | sid:1000001; |
| | **rev** (Mandatory) | Revision; the version number of the rule. Increment after any change. | rev:1; |
| **Payload** | **content** | Searches for specific data within the packet payload. | content:"/etc/shadow"; |
| | **pcre** | Perl Compatible Regular Expression matching for complex patterns. | pcre:"/useragent/i"; |
| **Non-Payload** | **flags** | Checks for specific TCP flags (e.g., F, S, R, P, A, U). | flags:S; (checks for SYN flag) |
| **Threshold** | **threshold/ detection_ filter** | Controls how often an alert can fire to prevent alert spamming. | detection_filter: type limit, track by_src, count 5, seconds 60; |

## 3. Lab Environment Setup (VirtualBox)

For Snort to detect traffic between other machines (Kali -> Metasploitable), the network must allow packet visibility across machines.

**Network Mode:** Set all three VMs to the same "Internal Network" (e.g., named SnortLab).

**Promiscuous Mode:**

Go to the Ubuntu VM Settings > Network > Advanced.

Set Promiscuous Mode to "Allow All".

## 3.1: System Preparation and Interface Configuration

1. Log in to your Ubuntu VM.
2. Update the system and install necessary networking tools:

```
sudo apt update
sudo apt upgrade -y
sudo apt install net-tools -y
```

3. Identify your Network Interface: Find the name of your primary network interface (e.g., enp0s1, eth0, etc) by typing the following command:

```
ip a
```

**Note: Replace enp0s1 with your interface name in all subsequent commands.**

4. Enable Promiscuous Mode (OS Level): This command allows the network card to accept all traffic on the segment, not just traffic addressed to its own MAC address.

```
sudo ip link set enp0s1 promisc on
```
**(Replace enp0s1 with your network interface name)**

5. Verification: Run the command 'ip a' again. You should see the flag PROMISC associated with your interface.

## Step 3.2: Installing and Initializing Snort

1. Install Snort:

```
sudo apt install snort -y
```

2. Configuration Prompts: During installation, you may be prompted:

“Interface: Enter your primary interface name (e.g., enp0s1).”

3. Enter your lab network subnet (e.g., 10.0.2.0/24). This sets the initial HOME_NET value.

### Step 3.3: Configuring snort.conf

The main configuration file is /etc/snort/snort.conf.

1. Backup the original file:
sudo cp /etc/snort/snort.conf /etc/snort/snort.conf.bak

2. Edit the configuration file:
sudo nano /etc/snort/snort.conf

3. Modify the following variables:

a. Set the Network to Protect: Find ipvar HOME_NET any. Change any to your subnet.

ipvar HOME_NET 10.0.2.0/24

b. Set External Network: Find ipvar EXTERNAL_NET any. Change to:

ipvar EXTERNAL_NET !$HOME_NET
(This tells Snort "everything not in my home network is external".)

## 4. Using Snort to Detect & Alert

We will use Snort in Console Mode first to see alerts in real-time.

**A. Using Predefined Rules (Community Rules)**

Snort includes predefined rule sets (e.g., scan.rules) that detect common attack patterns such as port scans and reconnaissance.These rules may be disabled by default.

1. Open snort.conf:
sudo nano /etc/snort/snort.conf
2. Scroll to the bottom to the include section.
3. Ensure the following lines are uncommented (by removing #):

include $RULE_PATH/local.rules
include $RULE_PATH/scan.rules
local.rules: contains user-defined custom rules
scan.rules: Built-in rule set for detecting scanning activity

**B. Creating Custom Rules**

We will write custom rules to detect the specific Nmap scans. Open your local rules file:

sudo nano /etc/snort/rules/local.rules

Add the following rules:

**Rule 1: Detect Ping flood**

alert icmp any any -> $HOME_NET any (itype:8; detection_filter:track by_src, count 20, seconds 5; threshold:type limit, track by_src, count 1, seconds 5; msg:"[Lab] ICMP Ping Flood Detected"; sid:1000001; rev:1;)

**Explanation:** Detects ICMP Echo Request packets (type 8) sent to the HOME_NET, and triggers an alert when 20 or more packets from the same source are detected within 5 seconds window and once triggered, only show 1 alert every 5 seconds

**Rule 2: Detect TCP Connect Scan (Flags)**.

Nmap -sT or normal connections often look like standard traffic, but we can detect high volumes or specific ports. For the lab, let's detect access to a specific port (e.g., SSH port 22).

alert tcp any any -> $HOME_NET 22 (flags:S; msg:"[Lab] SSH Connection Attempt"; sid:1000002; rev:1;)

**Explanation:** Detects TCP packets with the SYN flag set (connection initiation) targeting port 22 (SSH) on the HOME_NET, indicating an attempt to establish a connection to the SSH service.

**Rule 3: Detect Nmap SYN Scan.**

SYN scans send a SYN packet. If we see too many SYN packets in a short time, it's a scan.

alert tcp any any -> $HOME_NET any (flags:S; flow:stateless; detection_filter:track by_src, count 20, seconds 10; msg:"[Lab] Possible TCP SYN Scan"; sid:1000003; rev:1;)

**Explanation:** Detects TCP packets with the SYN flag set directed toward the HOME_NET and

triggers an alert when 20 or more such packets are received from the same source within 10 seconds, indicating a potential SYN-based port scan.

**Rule 4: Port Scan (Multi-port Detection)**

alert tcp any any -> $HOME_NET any (flags:S; flow:stateless; detection_filter:track by_src, count 50, seconds 5; msg:"[Lab] Possible Port Scan Activity"; sid:1000004; rev:1;)

**Explanation:** Detects a high volume of TCP SYN packets sent from a single source to the HOME_NET and generates an alert when 50 or more packets are observed within 5 seconds, indicating aggressive port scanning across multiple ports.

## C. Final Validation and Test Run

1. Validate the Configuration: This command checks your changes for syntax errors. Snort cannot run until this is successful.
sudo snort -T -c /etc/snort/snort.conf -i enp0s1
You must see: "Snort successfully validated the configuration!"
2. Run Snort in Sniffer Mode: This verifies the interface is capturing traffic.
sudo snort -vde -i enp0s1
3. Now, from Kali, ping the Metasploitable VM. You should see ICMP packet data scroll by, proving Snort is receiving traffic. Press Ctrl+C to stop it.

## Execution & Verification

1. Start Snort on the Ubuntu (NIDS) machine:
sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s1
On the Ubuntu server, start Snort to display alerts to the terminal (-A console) using your config (-c) and interface (-i). The -q flag is "quiet" mode, suppressing startup text so you only see alerts Snort will start and wait for incoming traffic.
2. From the Kali (attacker) machine, generate test traffic:
ICMP Ping Test: hping3 --icmp --flood 10.0.2.5
Expected alert on Snort: [Lab] ICMP Ping Flood Detected
3. TCP Connect Scan (SSH - Port 22): nmap -sT -p 22 10.0.2.5
Expected alert: [Lab] SSH Connection Attempt
4. SYN Scan: nmap -sS -F 10.0.2.5
Expected alert: [Lab] Possible TCP SYN Scan
or from built-in rules: SCAN nmap TCP scan detected
5. If you see the alerts, your NIDS is successfully sniffing "promiscuous" traffic from the network, and custom and built-in rules are functioning correctly.

6. If you only see alerts when you scan the Ubuntu IP but not the Metasploitable IP, your VirtualBox Promiscuous settings are incorrect.